

Nimbus — The Concepts Behind It

Version: 1.0 **Author:** Sujal Kumar Singh **Last updated:** 2026-08-09

The background knowledge you need before the other documents make sense. No system design here — just what the words mean and how email actually works.

Read this first, then `OVERVIEW.md`.

1. What a domain actually is

- A **domain** is a name you rent. `acme.com`. You pay a registrar (GoDaddy, Namecheap, Cloudflare) roughly \$10 a year.
- Renting it means you control **what that name points to**.
- The internet does not understand names. It only understands numbers — **IP addresses**, like `52.14.3.9`.
- **DNS** (Domain Name System) is the phone book that turns names into numbers.

DNS holds different records for different jobs

`acme.com`

|

├ A record → 52.14.3.9 "the WEBSITE is here"

├ MX record → mail.nimbus.io "the EMAIL goes here"

└ TXT record → "... " notes, used for anti-spam

MX means *Mail eXchange*. It is the single most important record for us.

The website and the email can live on completely different servers. `acme.com`'s website might be on Squarespace while its email is on Nimbus. They are separate records pointing at separate machines. Most people assume they are the same thing. They are not.

What this means for Nimbus

When a reseller provisions `acme.com` with us, we create the mailboxes immediately — but **no mail will arrive** until the owner of `acme.com` edits their DNS and points the MX record at our server.

That is what the `domain.verified` column is for. Provisioned is not the same as live.

2. How an email actually travels

There is **no central post office**. Mail servers talk directly to each other.

```
sujal@acme.com writes to ravi@globex.com
```

1. Sujal's mail app hands the message to acme's outgoing server
2. That server asks DNS: "what is the MX record for globex.com?"
DNS answers: "mail.globex.com, which is 91.2.3.4"
3. It opens a connection to 91.2.3.4 on port 25
4. The two servers have a short conversation (see section 3)
5. The receiving server stores the message
6. Ravi's mail app asks that server for his new mail later

Nimbus is step 5. We are the **receiving** server. We never do step 1 to 4 — that is the sending side, and it is an explicit non-goal.

Ports 25 and 2525

A **port** is a numbered door on a server. Port 25 is the agreed door for mail between servers — every mail server in the world knows to knock there.

- **Port 25** is the real one.
- **Port 2525** is an unofficial alternative used for development.
- Cloud providers **block port 25 by default** on new accounts, because spammers abuse it. AWS takes several days to unblock it after you ask.

So we develop on 2525 and request the unblock early.

3. The SMTP conversation, and what `RCPT TO` means

SMTP = Simple Mail Transfer Protocol. It is a conversation in plain text. You could type it by hand.

```

them: HELO mail.globex.com      "hello, I am globex's mail server"
us:    250 OK                    "hello, go ahead"

them: MAIL FROM:<ceo@globex.com> "this message is from this person"
us:    250 OK                    "fine"

them: RCPT TO:<sujal@acme.com>   "please deliver it to this person"
us:    250 OK                    "yes, I accept that recipient"
      — or —
us:    550 No such user          "no. I refuse this one."

them: DATA                      "here comes the actual message"
them: Subject: Q3 numbers
      <the whole email>
      .
us:    250 OK                    "received. it is mine now."

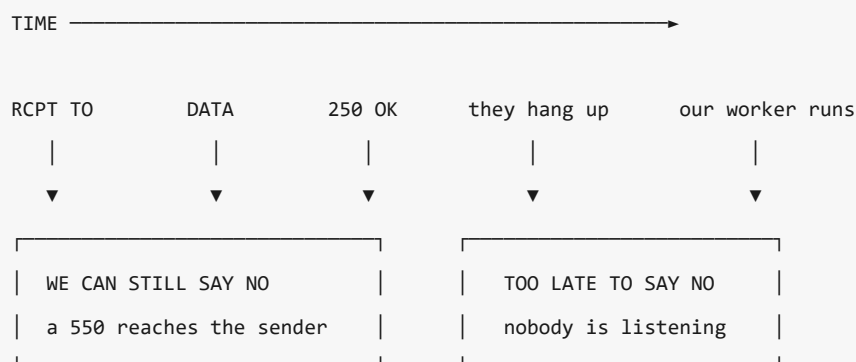
them: QUIT

```

Reading it

- **Every reply is a number.** 2xx = fine. 4xx = try again later. 5xx = refused, permanently.
- **MAIL FROM** names the sender. **RCPT TO** — short for *recipient to* — names **one** person who should receive it. It is repeated once per recipient.
- One message to 40 people means 40 **RCPT TO** lines in the same conversation.

Why **RCPT TO** is the moment everything hinges on



Once we answer **250 OK** to **DATA**, we have taken responsibility for that message. The sender hangs up and goes away. There is no longer anyone to tell.

So **every check that could reject mail has to happen at RCPT TO**. That single fact shaped a large part of this system: it is why our receiver checks the address before accepting, and why the routing chain's last step is "log and drop" rather than "bounce".

4. Why Redis, and what it is doing here

What Redis is

- A **key-value store** — you save a value under a name, and fetch it by that name.
- It keeps everything **in RAM**, not on disk. That is why it answers in about **0.1 milliseconds** instead of the 1–5 milliseconds a database takes.
- The trade: RAM is expensive and its contents vanish if the machine restarts. So you only put things in Redis that you can **rebuild from somewhere else**.

But Postgres already knows the answer. Why not just ask it?

Postgres does have the mailbox list. Four reasons we do not ask it at `RCPT TO` :

Reason	Detail
Volume	Every recipient of every message is one lookup. One email to 40 people = 40 lookups, in one conversation, while the sender waits.
Connections	The receiver holds up to 100 open SMTP connections, each able to ask about 100 recipients. If every one grabbed a database connection, a pool of 10 would be exhausted immediately.
Blast radius	The receiver is our front door to the internet. If the database is briefly slow, mail should still be accepted, not rejected.
Speed matters here specifically	The sender is waiting on the line. Slow answers at <code>RCPT TO</code> make other mail servers time out and retry.

And the risk is acceptable **because our Redis data is rebuildable**. On every API startup we regenerate the whole address list from Postgres. Redis is a cache. Postgres is the truth.

The four jobs Redis does in Nimbus

Job	Redis structure	Why that structure
"Does this address exist?"	SET — <code>valid_addresses</code>	A set holds no duplicates, and "is X in here?" is instant no matter how many millions are in it
"Have we stored this file before?"	plain key	The dedup check, run on every attachment
"Wake this message at 09:00"	sorted set	Every item carries a score. Use the time as the score and you have a timer queue for free.
"Don't upload this file twice at once"	lock key	Two workers receiving the same attachment at the same second

5. API keys, tokens, and why there are two kinds

An API key is exactly like a ChatGPT API key

Yes — same idea, same shape.

- It is a **long random string that says "I am this account"**.
- It is a password for a **program**, not a person.
- You send it on every request in a header:

```
Authorization: Bearer vHQmn0T2izjUUzpMVHRQXGtty...
```

OpenAI's looks like `sk-proj-...`. Stripe's looks like `sk_live_...`. Ours looks like a random 43-character string. Identical concept.

Why not just give the program a username and password? Because a key can be revoked and reissued on its own, without changing a human's login, and it can be given limited permissions. Also, a program storing a human's password is a bad idea.

We never store the key itself — only a hash of it. That is why `create_reseller.py` prints it once and says it cannot be recovered. If we stored it and were breached, every reseller's key would leak.

Nimbus has two different callers, so two different credentials

	Reseller	Mailbox user
Who	A company's server	A human in a browser
Credential	API key	Password, exchanged for a token
How long it lasts	Until revoked	24 hours
Example	MailHost India creating 500 mailboxes	Sujal reading his inbox

Why the human gets a token instead of sending their password every time

- The password would otherwise be sent on **every single request** and stored in the browser the whole time.
- A token **expires by itself**. A stolen one stops working.
- A token can carry limited permissions; a password always carries all of them.

What a JWT actually is

JWT = JSON Web Token. Three parts joined by dots:

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiI3YWlwMWVmZiJ9 . 4pQ8x_kZ...
```

header

payload

signature

header which signing method was used

payload the claims: "this is mailbox 7ab01aff", "expires 09:00 tomorrow"

signature proof that WE issued it

The payload is not encrypted. Anyone holding the token can read it — it is only base64 encoded. Paste one into jwt.io and you will see its contents in plain text.

What the signature gives you is **tamper-proofing**, not secrecy. Change one character of the payload and the signature no longer matches, so we reject it.

The practical rule: **never put anything private inside a JWT**. A mailbox id is fine. An email address is fine. A password or an internal note is not.

6. Who the customer actually is

We do **not** sell mailboxes to people. We sell to **resellers**.

```
NIMBUS (us – the engine)
```

```
|
```

```
└─ MailHost India (a reseller)
```

```
|
```

```
└─ acme.com (their customer) └─ sujat@acme.com
```

```
|
```

```
└─ globex.com (their customer) └─ ravi@globex.com
```

```
|
```

```
└─ CloudDesk UK (another reseller)
```

```
└─ ...
```

- A reseller signs up businesses, brands the product as their own, and calls our API.
- Their customers never know Nimbus exists.
- **This is why `reseller_id` is on almost every table.** Two resellers must never see each other's data, and — importantly — attachments are deduplicated *within* a reseller, never across them.

7. Mailbox, alias, shared mailbox, catch-all

Four things that all look like email addresses but behave differently.

Thing	Example	What it is
Mailbox	<code>suja1@acme.com</code>	A real inbox. Stores mail. Has a login.
Alias	<code>sales@acme.com</code> → <code>suja1</code>	A nickname. Stores nothing, just forwards to a real mailbox.
Shared mailbox	<code>support@acme.com</code>	One inbox several people can read. No login of its own.
Catch-all	<code>anything @acme.com</code>	A safety net. Mail to an address that does not exist lands here instead of being refused.

8. The words, in one table

Word	Plain meaning
Domain	A rented name like <code>acme.com</code>
DNS	The phone book turning names into numbers
MX record	The DNS entry saying "mail for this domain goes here"
IP address	A machine's number on the internet
Port	A numbered door on a machine. 25 = mail.
SMTP	The plain-text conversation mail servers have
MAIL FROM	The SMTP line naming the sender
RCPT TO	The SMTP line naming one recipient — the last moment we can say no
250 / 550	SMTP for "fine" and "refused"
MIME	The format inside an email that lets it carry text and files together
Redis	An in-memory store that answers in fractions of a millisecond
SET (Redis)	A bag with no duplicates; "is X in here?" is instant
Sorted set (Redis)	A set where each item has a score. Score = time gives you timers.
API key	A long random string identifying a program, like a ChatGPT key
Bearer token	Anything sent as <code>Authorization: Bearer ...</code>
JWT	A signed token: readable by anyone, forgeable by nobody
Hash	A one-way fingerprint. Easy to compute, impossible to reverse.
Reseller	The company that resells our email under their own brand

9. Where to go next

You want	Read
What we are building and why	OVERVIEW.md
The whole system drawn	ARCHITECTURE.md
The database explained	DATABASE.md
Exact columns and endpoints	HLD.md